

Aspectos básicos de JavaScript

Esta guía abarca la sintaxis fundamental, los conceptos estructurales, la programación orientada a objetos, el formato de intercambio JSON y el modelo de funcionamiento de JavaScript para el desarrollo web moderno.

Índice

- [1. Modos de Carga y Ejecución de Scripts en HTML](#)
 - [2. Declaración de Variables y Tipos de Datos](#)
 - [3. Operadores y Coerción de Tipos](#)
 - [4. Estructuras de Datos Primarias](#)
 - [5. Funciones y Ámbitos](#)
 - [6. Integración con la Web: DOM y CSS](#)
 - [7. Objetos, Clases y Herencia Prototípica](#)
 - [8. Formato JSON \(JavaScript Object Notation\)](#)
-

1. Modos de carga y ejecución de scripts en HTML

La forma en que se vincula un archivo de JavaScript en el documento HTML determina cómo el navegador descarga y ejecuta el código en relación con el procesamiento (*parsing*) del árbol DOM. Por defecto, cuando el navegador encuentra una etiqueta **<script src="...">**, interrumpe la construcción de la página HTML para descargar y ejecutar el archivo **antes** de continuar interpretando el resto del documento.

Para optimizar la carga y el rendimiento, la etiqueta **<script>** admite los atributos booleanos *async* y *defer*.

Comportamiento estándar (sin atributos)

El navegador detiene el análisis sintáctico del documento HTML (*parsing*), descarga inmediatamente el archivo JavaScript, lo ejecuta y, una vez finalizada la ejecución, reanuda el procesamiento del HTML. Puede provocar que la interfaz se quede congelada o que no se encuentren elementos del DOM si el script intenta acceder a ellos antes de que hayan sido creados.

```
<!-- Bloquea el procesamiento del HTML durante la descarga y ejecución -->  
<script src="script.js"></script>
```

Atributo *async* (asíncrono)

Indica al navegador que descargue el archivo de forma asíncrona en segundo plano sin interrumpir el procesamiento del HTML. Sin embargo, **en el instante preciso en que**

finaliza la descarga, el navegador pausa la interpretación del HTML para ejecutar el script .

- **Uso recomendado:** scripts independientes que no dependen de la estructura del DOM ni de otros scripts (por ejemplo, herramientas de analítica web, contadores o servicios externos).
- **Orden de ejecución:** no garantizado. Si hay varios scripts con *async*, se ejecutará primero el que termine de descargarse antes.

```
<!-- Carga asíncrona: se ejecuta tan pronto como finaliza la
descarga -->
<script src="analytics.js" async></script>
```

Atributo *defer* (diferido)

Le indica al navegador que descargue el archivo en segundo plano mientras continúa procesando y construyendo el árbol DOM. A diferencia de *async*, **la ejecución del script se posterga hasta que el parsing del HTML haya finalizado por completo** (justo antes del evento *DOMContentLoaded*).

- **Uso recomendado:** la mayoría de scripts de la aplicación que requieren manipular elementos del DOM o que dependen de otros scripts en un orden específico.
- **Orden de ejecución:** se garantiza que los scripts marcados con *defer* se ejecuten en el mismo orden estricto en que aparecen definidos en el documento HTML.

```
<!-- Carga en segundo plano: se ejecuta únicamente cuando el HTML
esté listo -->
<script src="app.js" defer></script>
```

Cuadro comparativo de carga y ejecución

Método de inclusión	Descarga del archivo	Ejecución del Script	Garantía de Orden
<script>	Bloquea el parsing HTML	Inmediata (bloqueante)	Según orden en el documento
<script async>	En segundo plano (asíncrona)	Tan pronto como descarga	Ninguna (según disponibilidad)
<script defer>	En segundo plano (asíncrona)	Al finalizar el parsing HTML	Según orden en el documento

En resumen:



Imagen sacada de <https://www.josefzacek.cz/blog/whats-the-difference-between-async-vs-defer-attributes/>

2. Declaración de variables y tipos de datos

JavaScript es un lenguaje con **tipado dinámico**, lo que significa que las variables no están ligadas a un tipo de dato específico, sino que el tipo se deduce según el valor asignado durante la ejecución.

Variables (const y let)

En JavaScript es posible declarar variables utilizando distintas palabras clave que definen su alcance y su mutabilidad a lo largo del programa:

- **const**: se utiliza para declarar constantes o variables cuyos valores no serán reasignados. Es la opción recomendada por defecto.
- **let**: se emplea para declarar variables con ámbito de bloque cuyo valor puede cambiar a lo largo del flujo del programa.
- **var**: es la forma tradicional y antigua de declarar variables. Se desaconseja su uso moderno debido a su alcance global/de función y a los problemas asociados al *hoisting*.

```
const maxIntentos = 3;  
let contador = 0;
```

Tipos Primitivos

JavaScript cuenta con un conjunto de tipos de datos primitivos que representan valores inmutables gestionados directamente por el lenguaje:

- **number**: representa valores numéricos, tanto enteros como números de coma flotante (estándar IEEE 754 de 64 bits).
- **string**: representa secuencias de caracteres empleadas para manipular texto.
- **boolean**: representa un valor lógico que solo puede ser true o false.
- **null**: representa la ausencia intencional de cualquier valor u objeto.
- **undefined**: indica que una variable ha sido declarada pero aún no se le ha asignado ningún valor.

- **symbol / bigint**: se utilizan para crear identificadores únicos inmutables y para manejar valores enteros numéricos de precisión arbitraria respectivamente.
-

3. Operadores y coerción de tipos

JavaScript es un lenguaje de **tipado débil**, lo que significa que el entorno de ejecución realiza conversiones automáticas e implícitas de tipos (proceso conocido como coerción) cuando se evalúan operaciones entre diferentes tipos de datos.

Igualdad estricta vs. débil

Para comparar valores en JavaScript, existen dos tipos de operadores de igualdad que se comportan de forma diferente ante la coerción de tipos:

- **=== (Igualdad estricta)**: evalúa si dos operandos son iguales comparando tanto su valor como su tipo de dato sin aplicar conversión implícita. **Es la norma recomendada de desarrollo.**
- **== (Igualdad débil)**: fuerza la conversión implícita de tipos antes de hacer la comparación, lo cual puede derivar en resultados inesperados e inconsistencias.

```
5 === "5"; // false (number vs. string)
5 == "5";  // true (convierte "5" a number)
```

4. Estructuras de datos primarias

JavaScript ofrece estructuras integradas para almacenar colecciones complejas de información y estructurar los datos dentro de las aplicaciones web.

Objetos

Los objetos son colecciones de pares clave-valor que permiten estructurar la información en entidades más complejas. Constituyen el pilar fundamental del lenguaje:

```
const persona_usuario = {
  nombre: "Bruce",
  rol: "ADMIN",
  activo: true
};

// Acceso a propiedades
console.log(persona_usuario.nombre);
console.log(persona_usuario["rol"]);
```

Arrays

Los arrays son listas ordenadas de elementos indexados secuencialmente a partir de cero. Proveen múltiples métodos funcionales integrados para procesar sus elementos de forma limpia:

```
const tecnologias = ["HTML", "CSS", "JavaScript"];
```

```
// Métodos comunes de iteración
const mayusculas = tecnologias.map(tech => tech.toUpperCase());
const filtrados = tecnologias.filter(tech => tech.length > 3);
```

5. Funciones y ámbitos

En JavaScript, las funciones se tratan como **ciudadanos de primera clase**, lo que significa que pueden ser asignadas a variables, pasadas como argumentos a otras funciones y retornadas como valores.

Funciones declarativas vs. flecha (*Arrow Functions*)

Existen diferentes formas de definir funciones según las necesidades del contexto y la sintaxis deseada.

```
// Declaración tradicional de función
function sumar(a, b) {
  return a + b;
}

// Función flecha (Arrow Function) con sintaxis compacta
const multiplicar = (a, b) => a * b;
```

Clausuras (*Closures*)

Una clausura es la combinación de una función y el entorno léxico en el que fue declarada. Esto permite que una función recuerde y acceda a las variables de su ámbito externo incluso después de que la función contenedora haya finalizado su ejecución.

6. Integración con la Web: DOM y CSS

JavaScript actúa como el motor de comportamiento dinámico dentro del navegador, conectando las capas de estructura (HTML) y presentación (CSS).

Manipulación del Document Object Model (DOM)

A través del DOM, JavaScript puede seleccionar elementos HTML, escuchar la interacción del usuario y modificar la estructura o los estilos en tiempo real.

```
// Selección de elementos del documento
const boton = document.querySelector("#btn-enviar");
const tarjeta = document.querySelector(".card");

// Registro de eventos para modificar la vista
boton.addEventListener("click", () => {
  // Alteración de clases CSS (práctica recomendada sobre
  // modificar estilos en línea)
  tarjeta.classList.toggle("destacado");
});
```

7. Objetos, clases y herencia

Aunque JavaScript utiliza internamente una **herencia basada en prototipos**, la sintaxis moderna de clases (*class*) introducida en ES6 permite estructurar la programación orientada a objetos de una manera más clara y legible.

Definición de clases y métodos

Una clase funciona como un molde que define el constructor, las propiedades y los métodos que compartirán las instancias creadas a partir de ella:

```
class User{
  // Propiedad privada (ES2020)
  #password;

  constructor(name, email, password) {
    this.name = name;
    this.email = email;
    this.#password = password;
  }

  // Método de instancia
  autenticar(inputPassword) {
    return this.#password === inputPassword;
  }

  // Métodos de acceso (Getter)
  get getProfile() {
    return `${this.name} (${this.email})`;
  }
}

// Instanciación del objeto
const user1 = new User("Bruce", "bruce@ejemplo.com",
"secreta1234");
console.log(user1.getProfile()); // Bruce (bruce@ejemplo.com)
```

Herencia (*extends* y *super*)

La herencia permite crear nuevas clases que extienden las capacidades de una clase base, reutilizando sus métodos y propiedades mediante la palabra clave *super*.

```
class Administrator extends User{
  constructor(name, email, password, permissions = []) {
    super(name, email, password); // Invocación al constructor de
la clase base
    this.permissions = permissions;
  }

  hasPermission(permission) {
    return this.permissions.includes(permission);
  }
}

const admin1 = new Administrator("Lucas", "lucas@ejemplo.com",
```

```
"adminPass", ["DELETE_USER", "WRITE"]);  
console.log(admin1.hasPermission("DELETE_USER")); // true
```

8. Formato JSON (JavaScript Object Notation)

JSON es un formato estándar de texto plano de intercambio de datos, caracterizado por ser liviano y fácil de interpretar tanto por humanos como por computadoras.

Reglas del formato JSON vs. Objetos JavaScript

Aunque deriva de la sintaxis literal de objetos de JavaScript, JSON es un formato independiente que sigue reglas más estrictas.

- **Claves:** en JSON todas las claves deben ser cadenas de texto strictly delimitadas por comillas dobles ("clave": valor).
- **Valores permitidos:** solo se permiten cadenas, números, booleanos, arreglos, objetos nulos (null) u otros objetos JSON. No es posible incluir funciones, métodos ni el valor undefined.

Métodos del objeto global JSON

El entorno de JavaScript incluye un objeto global JSON con utilidades para transformar datos entre objetos en memoria y cadenas de texto.

```
// Objeto definido en código JavaScript  
const configuracionServidor = {  
  puerto: 8080,  
  entorno: "produccion",  
  habilitado: true  
};  
  
// 1. Serialización: convierte un objeto de JavaScript a una  
// cadena de texto en formato JSON  
const jsonString = JSON.stringify(configuracionServidor);  
console.log(typeof jsonString); // "string"  
  
// 2. Deserialización: convierte una cadena de texto en formato  
// JSON a un objeto de JavaScript  
const jsonRecibido =  
  '{"puerto":3000,"entorno":"desarrollo","habilitado":false}';  
const objetoParseado = JSON.parse(jsonRecibido);  
console.log(objetoParseado.puerto); // 3000
```